CMU 15-441/641: Networking and the Internet
Project 2: TCP in the Wild

TAs: Ayushi Bansal, Yujing Wang, Yiran Lei, Isabel Suizo, Mingjun Li
Last update: October 6, 2026

Checkpoint 1 due: 6:00 pm on October 7th, 2026
Checkpoint 2 due: 6:00 pm on October 28th, 2026
Lab due: 6:00 pm on November 6th, 2026

1 Introduction

TCP is a transport layer protocol that enables processes on different devices to communicate reliably. TCP's beauty stems from its ability to give applications a simple abstraction with strong guarantees, even though the underlying network only provides best-effort delivery. Best effort delivery means that packets can be lost, duplicated, reordered, and that there is no guarantee that the bandwidth available will remain the same (due to changes in the network itself or interference from other traffic). TCP on the other hand assures applications that their data will be delivered in order, uncorrupted, and that transmissions will be paced to avoid overwhelming the network (congestion control) or the application on the other end of the connection (flow control). It also aims to be fair, trying to give an equal share of bandwidth to multiple senders that share the same link.

In this project, your goal is to implement the underlying mechanisms that are the basis of TCP's strong guarantees. You will focus on implementing an algorithm that is very similar to TCP Reno. However, TCP has evolved since Reno, which is now just one of many congestion control algorithms (CCAs) in use. Companies use different CCAs depending on their context. For example, one version may be appropriate inside of datacenters while another version may be preferable for serving web content on the Internet. For web content, the most common algorithm—and the default in Linux—is called Cubic. Akamai, the largest content distribution network in the world, uses a proprietary TCP called FastTCP. In Lab, you will have the option to implement your own CCA for real-world challenges.

With this project, you will gain experience in:

- Building non-trivial computer systems in a low-level language (C).
- Building interoperable programs for use on the Internet.
- Designing end-to-end systems when the underlying network is fundamentally unreliable.
- Analyzing an algorithm's performance, designing ways to improve these metrics, and testing those improvements.

This project consists of two checkpoints and one lab. In the first, you will implement the TCP three-way handshake as well as TCP's sliding window algorithm. In the second, you will implement fast recovery, flow control, and congestion control. In the lab, you will further upgrade your implemented transport protocol to address real-world challenges. This writeup only contains information for the checkpoints, and we will release lab writeup later. **You may complete this project individually or in a group of at most two students.**

2 CMU-TCP

You will implement CMU-TCP, a simple reliable transport protocol. Different from a traditional TCP implementation, CMU-TCP segments are carried inside UDP payloads, this allows you to implement CMU-TCP from user space

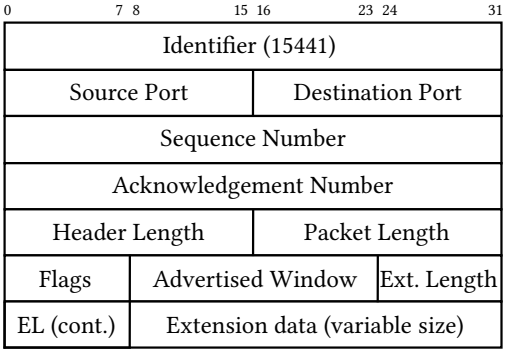


Figure 1: CMU-TCP header format.

using UDP sockets and benefit from UDP's existing multiplexing/demultiplexing capabilities. As in traditional TCP, CMU-TCP connects two end points, an initiator and a listener. While a CMU-TCP header shares many fields with traditional TCP, its structure is different. Figure 1 shows the structure of a CMU-TCP header, with each field described next:

- *Identifier*: All CMU-TCP packets start with an identifier that MUST be set to 15441.
- *Source Port*: Identifies the source port (matches the UDP source port in the header below).
- *Destination Port*: Identifies the destination port (matches the UDP destination port in the header below).
- *Sequence Number*: The sequence number of the first byte in this segment, unless if `SYN` is set. If `SYN` is set, this number is the initial sequence number (ISN). The first data sent, will have ISN+1 as the sequence number.
- *Acknowledgment Number*: Valid when the `ACK` flag is set. Identifies the next sequence number that the sender expects.
- *Header Length*: Length of the CMU-TCP header in bytes (25 bytes with no extension data).
- *Packet Length*: Length of the CMU-TCP packet (payload + header) in bytes.
- *Flags*: Only 3 out of 8 bits are used (from left to right starting at 0):
  - `SYN` (bit 4): Synchronize sequence numbers. Used by both sides at the beginning of the connection.
  - `ACK` (bit 5): Acknowledgment number is significant.
  - `FIN` (bit 6): Sender has no more data to send.
- *Advertised Window*: Number of bytes that the sender can accept in its window.
- *Extension Length*: Length of the "Extension Data" field in bytes.
- *Extension Data*: Arbitrary extension data. (You may use this when developing your own CCA in the Lab.)

As a starting point, you will receive starter code that implements CMU-TCP using stop-and-wait transmission. However, as we learned in class, stop-and-wait is very inefficient because it can only transmit a single packet per RTT. The starter code also lacks connection setup and termination, congestion control and flow control. In **Checkpoint 1**, you will augment the starter code with a handshake and window, so that multiple packets can be sent before the next ACK arrives. For Checkpoint 1, this window will be fixed and set at compile time. Optionally, you can also implement connection termination for bonus points. In **Checkpoint 2**, you will improve on the performance of your implementation by automatically choosing an appropriate window size. You will need to implement flow control and congestion control, using a basic version of TCP Reno as the congestion control algorithm. Checkpoint 2 also requires that you run some experiments to test a hypothesis. You should make sure to finish your implementation early so that you have time to run the experiments.

You will build CMU-TCP using UDP sockets, crafting packets and transmitting them yourself. UDP will not re-transmit lost packets, and UDP has no control over when a packet is sent (i.e., the sending throughput). You must augment UDP with these features yourself. As both sides (initiator and listener) can send and receive, you will need to track a lot of data and information. It is important to write down everything that each side knows while working on your implementation and to utilize interfaces to keep your code modular and reusable. A practical guide to implementing TCP is found in the textbook in Sections 5.2 [1] and 6.3 [2]. *Please read these textbook sections before starting the project!* If in doubt or faced with ambiguity, prefer explanations from the textbook.

2.1 Starter Code and Implementation Details

The starter code is hosted on GitHub: https://github.com/computer-networks/cmu-tcp. See README.md for a description of the files in the starter code and instructions on how to run it. **You may use AI agents to interpret materials, design, debug, and test. If you do, load .agent/P2LEARNINGGOALS.md and .agent/AGENT.md into the agent's instructions — it tells the agent not to implement the core mechanisms of this project, which you are expected to write and explain yourself**.

The API exposed to applications is defined in cmu_tcp.h and consists of four core functions whose signatures should not be changed (cmu_socket, cmu_close, cmu_read, and cmu_write). An application requests a new CMU-TCP socket by calling the cmu_socket function. The application specifies whether it is a listener (e.g., a server) or an initiator (e.g., a client). For listener sockets, the application specifies the port to listen on. For initiator sockets, the application specifies the IP and port number of the *listener* socket. After initializing the socket, the application can use cmu_write to transmit data and cmu_read to receive data. cmu_read also accepts flags that define whether it should block when there is no data to be read (blocking) or return immediately with no data (non-blocking). When the application is done sending data, it is responsible for calling cmu_close to terminate the socket.

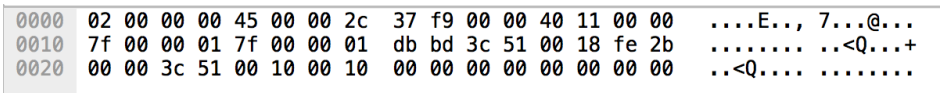
While the functions defined in the API are implemented in cmu_tcp.c, which runs on the same thread as the application, most of CMU-TCP's core logic actually runs in a separate backend thread. This is important as TCP must be able to work *independently* from the application (i.e., receiving, acknowledging and retransmitting packets). As such, the logic implemented in cmu_tcp.c is simply responsible for communicating with the backend thread and most of your implementation should be done at backend.c (functions handle_message, single_send, begin_backend).

The starter code also includes two sample applications (server.c and client.c) that use CMU-TCP to communicate. You can use and adapt these applications to test our code. We also provide simple automatic tests written in Python that you can use as a basis to implement more refined tests.

We will use grading.h to help us test your code. We may change any of the values for the variables present in the file to make sure that you are not hard-coding something elsewhere in your code. Namely, we will be changing the packet length and the initial window variables. Additionally, for CP2, we will be monitoring and reviewing your congestion window to ensure that it is consistent with TCP Reno. This will be manually graded after the due date.

All multi-byte integer fields must be transmitted in network byte order. ntoh(), hton(), and similar functions are important. We also provide many helper functions to get and set different packet fields, automatically dealing with endianness. Refer to cmu_packet.h for their signatures and cmu_packet.c for their implementation. You must also be careful when comparing sequence number as they might wrap around. When comparing sequence numbers you may use one of the provided helper functions (before, after, between) in cmu_packet.h. You should also make sure that IDENTIFIER is always set to 15441, even if you are taking 15-641 (the scripts rely on this). You are not allowed to change the structure of the header, with the exception of the extension data which you may want to modify when implementing your own CCA in Lab. Additionally, packet length (plen) cannot exceed MAX_LEN to prevent packets from being broken into parts.

You can verify that your headers are sent correctly using Wireshark or tcpdump (Section 4.3), which allow you to inspect packet data, including the full Ethernet frame. When viewing a packet, you should see something similar to the image below; in this case the payload starts at 0x0020. The IDENTIFIER (15441) shows up in hexadecimal as 0x00003C51.



3 Evaluation

This project is broken down in two checkpoints to help you maintain pace with required work. Your grade for the first checkpoint will be based solely on an implementation component, which is determined by manual evaluation as well as a suite of automated test cases on Gradescope.

For the second checkpoint, in addition to have an implementation component, there will be an experimental component, where you will hypothesize about, experiment with, and answer questions relating to your CMU-TCP implementation. Thus, for the second checkpoint, you need to submit both the source code and a PDF containing your answers. You should start your implementation early to also have time to run experiments and write the report.

3.1 Checkpoint 1

In this checkpoint, you will implement windowing so that multiple packets can be 'in the pipe' simultaneously. To do so, you also need to implement the TCP three-way handshake. This ensures that even if the first packet is lost, the data will be transferred reliably. Optionally, you can also implement connection teardown for bonus points.

Your implementation should:

1. Perform the TCP three-way handshake [3].
2. Initialize sequence numbers randomly and synchronize them during the handshake.
3. Transmit multiple packets before an ACK by using a fixed window size (you must set the window size to `CP1_WINDOW_SIZE` bytes) [4].
4. Retransmit packets after a timeout using Go-Back-N (timeout should be set to `DEFAULT_TIMEOUT`).
5. [Optional] Perform connection teardown [3].

You should use the provided script (`prepare_submission.sh`) to create an archive with your submission (`handin.tar.gz`). Make sure that all the files that are relevant to the submission are committed before you run the script.[1] You can then submit this archive to Gradescope. Your implementation will be automatically graded using the following rubric:

Category	Weight	Criteria (points)
Initiator handshake	20%	*Autograded:* • Sends the first SYN packet properly • Retransmit SYN if the first SYN is lost • Rejects malformed SYN-ACK packet from listener • Receives data packets properly after handshake finishes
Listener handshake	30%	*Autograded:* • Does not respond to invalid SYN packets • Responds to valid SYN packets with a valid SYN-ACK packet • Retransmits SYN-ACK packets on loss • Properly handles invalid ACK packets after a SYN-ACK • Receives data packets properly after handshake finishes
Reliability	30%	*Autograded:* • Retransmits data packets on timeout • Transfers a file reliably under lossless conditions • Transfers a file reliably under lossy conditions
Windowing, Sequence numbers	20%	*Autograded:* • Sender transmits multiple data packets at a time • Sender slides the window after receiving ACKs. • Sender correctly implements Go-Back-N upon packet loss • Initiator correctly synchronizes sequence number • Listener correctly synchronizes sequence number • Initiator initializes sequence number randomly • Listener initializes sequence number randomly
Teardown (Optional)	+5%	*Autograded:* • Sends FIN packet correctly • Sends a valid ACK packet after receiving a FIN-ACK • Retransmits FIN packet on timeout • One side can still send data after the other side starts teardown

[1] You may also consider tagging [12] your commit once you are done with the checkpoint, so that you can easily compare your CP1 solution with your CP2 solution.

3.2 Checkpoint 2

Now that you have implemented connection setup and windowing, it's time to improve the performance of your implementation by adding congestion control and flow control, which allows your window size to dynamically change. These two techniques are essential to ensure that the sender neither sends data too slow (thereby underutilizing the available bandwidth), or too fast (thereby overwhelming either the receiver or the network). **Both implementation and report are required for this checkpoint.**

3.2.1 Implementation Component

To determine the size of the sender's window, you will implement TCP Reno, as discussed in class. The sending window is limited by `min(`*"receiver advertised window," "congestion window"*`)`. As described below, you must demonstrate using graphs from real connections that your TCP Reno implementation uses Additive Increase under normal operation, and Multiplicative Decrease (when appropriate) under loss. Consult textbook Section 6.3 [2] for an in-depth discussion of what you should implement. Your implementation should:

1. Implement retransmission on duplicate ACKs (fast recovery): Another reason that loss recovery is slow is that the starter code relies on timeouts to detect packet loss. One way to recover more quickly is to retransmit whenever you see three duplicate ACKs. Implement retransmission on the receipt of three duplicate ACKs. Note that three duplicate ACKs means four ACKs acknowledging the same sequence number.

2. Implement flow control: Update your code to use the receiver's `AdvertisedWindow` as your maximum window size; this field is contained in the CMU-TCP header. You will need to modify your receiver to update `AdvertisedWindow` as it receives data.

3. Implement congestion control: You will need to add a new parameter, the congestion window: `cwnd`. The size of your sending window should now be the minimum of `cwnd` and the advertised window. You should additionally make sure that the total amount of data buffered for the application (unread data, both ordered and unordered bytes) is less than `MAX_NETWORK_BUFFER` (see details below).

To make sure that your code is robust you should test your new features with many different network settings using `tcconfig` (Section 4.2). `tcconfig` lets you set the network characteristics of your environment to help test various aspects of your implementation. For example, adding loss to the connection will help you to test reliable transmission.

A good way to quickly check that your implementation is reasonable is to transmit a large file and plot the sender's congestion window over time. The congestion window should exhibit the TCP sawtooth pattern. To get `cwnd`, you can instrument your code to log the status of your `cwnd`.
Figure 2 shows the full TCP Reno congestion control state diagram, for your reference. Below are the variables from `grading.h` that you must use in your code for this checkpoint. We will test your code by changing the values of these variables – you may also change these values to make sure that your code is robust but be aware that your changes to this file will be overwritten by the autograder. All of the following values are in bytes.

1. `WINDOW_INITIAL_WINDOW_SIZE`: Initial window size for slow start. In slow start, you should initially set `cwnd = WINDOW_INITIAL_WINDOW_SIZE`.

2. `WINDOW_INITIAL_SSTHRESH`: `ssthresh` value for congestion control. In slow start, you should initially set `ssthresh = WINDOW_INITIAL_SSTHRESH`.

3. `MAX_LEN`: Max packet length, including the header, for any packet. This value will not change.

4. `MSS`: Maximum segment size. This is the largest segment that can be transmitted without causing fragmentation.

5. `MAX_NETWORK_BUFFER`: Maximum number of bytes that the TCP implementation can hold/buffer for the application. This includes unread, ordered, and unordered bytes received on the network, and received by the application. Thus, the size of both your `sending_buf` and `received_buf` should be equal to `MAX_NETWORK_BUFFER`.

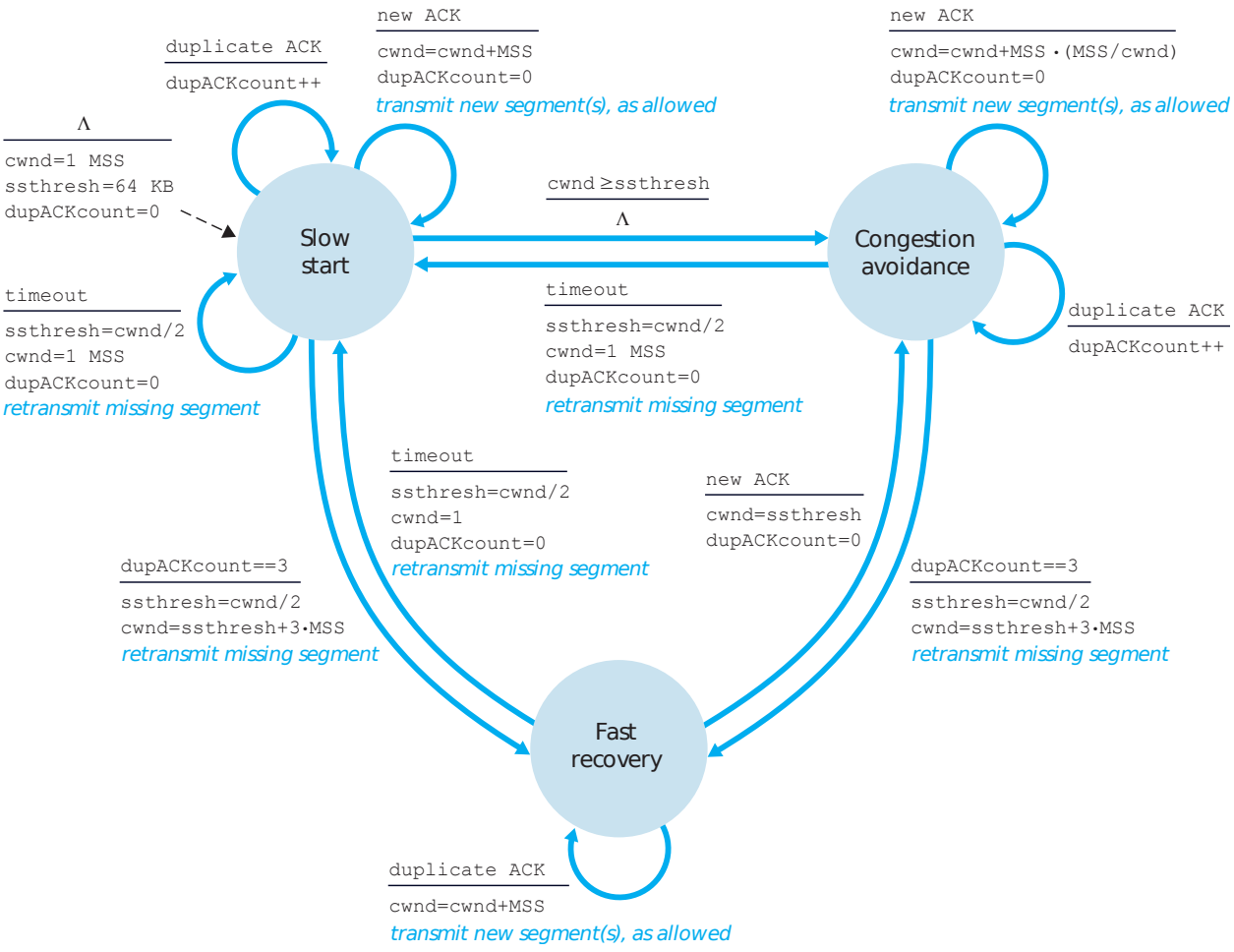


Figure 2: Congestion control state diagram for TCP Reno. (Extracted from *Computer Networking: A Top-Down Approach (Seventh Edition)* by Kurose and Ross [15].)

As in CP1, you should use the provided script (`prepare_submission.sh`) to create an archive with your submission (`handin.tar.gz`). Make sure that all the files that are relevant to the submission are committed before you run the script. You can then submit this archive to Gradescope. Your implementation will be automatically and manually graded using the following rubric:

Category	Weight	Criteria (points)
Regression Tests	10%	*Autograded:* • Listener retransmits SYN-ACK packets on timeout • Listener properly handles invalid ACK packets after SYN-ACK • Initiator properly handles bad SYN-ACK packets • Initiator receives data packets properly after handshake finishes • Transfers a file reliably under lossless conditions • Transfers a file reliably under minor lossiness • Transfers a file reliably under severe lossiness
Fast retransmission, flow control	20%	*Autograded:* • Sender correctly responds to changes in the receiver's advertised window • Advertised window is correctly reduced when the application does not consume data • A packet is retransmitted after three duplicate ACKs
Congestion control	20%	*Autograded:* • In slow start, the congestion window increases exponentially • In congestion avoidance mode, the congestion window correctly responds to loss • In congestion avoidance mode, the congestion window increases linearly • Sender returns to slow start on timeout
Congestion control report	20%	*Manually graded (create a PDF report and submit it to Gradescope):* • Include a graph in your report that shows the sender's congestion window (`cwnd`, in bytes) over time. Describe the congestion-control stages and explain how the window changes in each stage. You should develop a scenario that forces your implementation into a few 3-DupAck and 1-2 timeouts: transfer a large file—big enough that the transfer takes at least a few seconds—over a lossy network that you configure with `tcconfig` (Section 4.2). Report the network configuration and the file transfer size that you used. Make sure to label the following events in your plot: - Every point where a *3-DupAck* occurs - Every point where a *timeout* occurs - Label *slow start* - Label *congestion avoidance* - Label *fast recovery*
Mathis model evaluation	30%	*Manually graded (see Section 3.2.2 for the full description):* • Measures throughput over a range of loss probabilities, repeating each setting 10 times • Includes a plot of throughput against $1/\sqrt{p}$ with the linear regression line • Reports the value of C obtained from the regression • Reports the Pearson Correlation Coefficient between $1/\sqrt{p}$ and the measured throughput • Answers whether the data corroborate Dr. Mathis's hypothesis, with justification

We provide `utils/capture_packets.sh` and `utils/tcp.lua` to capture and inspect packets for debugging, as

described in Section 4.3. To plot the congestion window over time, **instrument your sender to record `cwnd` and congestion-control stages**.

3.2.2 Experimental Component

For the experimental component, perform the following evaluation and provide your answers in the report.

Hypothesis. In this checkpoint, we are going to give you a hypothesis formulated by Dr. Matt Mathis: that TCP Reno implementations obey the following throughput equation:

$$\text{throughput} = C \times \frac{1}{\sqrt{p}}. \tag{1}$$

Where C is a constant and p is the loss probability. The following measurements will help us evaluate whether or not this hypothesis is true.

Experiment. Set up two nodes on AWS, use one node as the sender and the other as the receiver. Use `tcconfig` (Section 4.2) to hold the bandwidth and delay fixed while you vary only the loss probability p. Before each run, configure both nodes as follows, substituting the loss probability for <p>:

```
# On the sender:
sudo tcset $IFNAME --rate 5Mbps --delay 20ms --loss <p> --overwrite

# On the receiver:
sudo tcset $IFNAME --rate 5Mbps --delay 20ms --overwrite
```

Applying a 20 ms one-way delay on both nodes gives a 40 ms RTT, and loss is applied only on the sender so that it affects the data path rather than the returning ACKs. Keep the rate and the delay at these values for every run so that any change in throughput comes from p alone. Sweep $p \in \{0.1\%, 0.2\%, 0.5\%, 1\%, 2\%, 5\%, 10\%\}$. For each value of p, transfer a large file and measure the duration of the transfer (make sure to adjust the file size so that your transfer takes at least a few seconds). Use the transfer duration and the file size to calculate the throughput. Repeat the same experiment 10 times for each loss probability. Create a line plot with $1/\sqrt{p}$ on the x axis and throughput on the y axis. Do linear regression and include the regression line on the same plot.

Inference. Based on the results of your experiments, we ask that you answer the following questions:

- Using your linear regression, what value did you find for C?
- Compute the Pearson Correlation Coefficient between $1/\sqrt{p}$ and the throughput that you measured.
- Does your data corroborate Dr. Mathis's hypothesis? Why or why not?

4 Resources

This section describes some of the recommended tools for developing and testing CMU-TCP.

4.1 Development Environment

For this project, you can do all of your development on your own machine using containers. You are also welcomed to keep using AWS for your development if that is more convenient. You should install Docker [11] and Vagrant [10] on the machine you plan to develop on. We have provided a `Vagrantfile` specifying two containers, a client and a server. The Vagrantfile is configured to map the repository's directory to the containers' `/vagrant/` directory. This means that any change you make to this directory, either from your computer or inside the containers will be propagated everywhere else. You can build and launch the containers by running `vagrant up --provider=docker` from the `cmu-tcp` directory. Once the containers finish starting up, you can log into any of the two containers using `vagrant ssh {client | server}`. The server and client are connected via a virtual network and have IP addresses `10.0.1.1` and `10.0.1.2`, respectively. But note that your code should work even if these IP addresses change.

Note: This project has only been tested on Linux environments, and not on Windows or WSL. We recommend doing the project on a virtual machine or on AWS if you don't have access to a Linux machine.

The following sections describe the tools that are already installed on the containers to help you test your code. You can also install any additional tools you need.

4.2 Control network characteristics with `tcconfig`

`tcconfig` [5] is installed on the containers to let you artificially control the characteristics of the virtual network that interconnects the containers. The initial default settings on the containers are a 20 ms delay on both containers (so the total RTT is 40 ms), and 100 Mbps bidirectional bandwidth with no loss. Running `tcshow $IFNAME` on the containers will show you these settings. You can set additional `tc` parameters by using `tcset`, for example `sudo tcset $IFNAME --rate 10Mbps --delay 20ms --loss 0.1% --overwrite` will set the rate to 10 Mbps, single-way delay to 20 ms, and loss probability to 0.1%.[2]

Refer to the references for more information on how to use `tcconfig` to simulate different network characteristics that will be useful for testing your code, including packet loss, reordering, and corruption.

4.3 Capture and analyze packets with `tcpdump` and `tshark`

`tcpdump` [6] and Wireshark (and its equivalent command line program: `tshark` [7]) are installed on the containers to allow you to capture packets sent between the containers and analyze them later. We provide the following files in the directory `project-2_15-441/utils/` to help with packet analysis (feel free to modify these if you want):

- `capture_packets.sh`: This is a simple program showing how to start and stop packet captures, as well as analyze packets using `tshark`. The `start` function starts a packet capture in the background. The `stop` function stops a packet capture. Lastly, the `analyze` function will use `tshark` to output a CSV file with header information from your TCP packets. See Section 4.4 for an example.
- `tcp.lua`: This is a Lua plugin so that Wireshark can dissect our custom CMU-TCP packet format [8]. `capture_packets.sh` shows how you can pass this file to `tshark` to parse packets. To use the plugin with the Wireshark GUI on your machine, you should add this file to Wireshark's plugin folder [9].

4.4 Analyzing a large file transfer

The starter code `client.c` and `server.c` will transmit a small file, `cmu_tcp.c` between the client and server. You should also test your code by transmitting a large file and inspecting the packet trace. The following commands create `capture.pcap`:

(In the *server* container) Start `tcpdump` and the server:

```
vagrant@server:/vagrant/project-2_15-441$ make
vagrant@server:/vagrant/project-2_15-441$ ./utils/capture_packets.sh start capture.pcap
vagrant@server:/vagrant/project-2_15-441$ ./server
```

(In the *client* container) Start the client:

```
vagrant@client:/vagrant/project-2_15-441$ ./client
```

[2] Note that `$IFNAME` is a variable with the name of the interface that we will use with CMU-TCP, it is automatically initialized by the Vagrant scripts that we provide. When running on AWS, make sure to set it the name of the interface that you want to use.

(In the *server* container) When the transfer finishes, stop the packet capture:

```
vagrant@server:/vagrant/project-2_15-441$ ./utils/capture_packets.sh stop capture.pcap
```

4.5 Running tests with `pytest`

There are many ways that you can write tests for your code. To help get you started, `pytest` [13] is installed on the containers and we provide some examples of basic tests in `tests/test_cp1.py`. Running `make test` will run these tests automatically. You should expand these tests or use a different tool to test your code. However, be sure to update the Makefile so that `make test` still runs your tests. As in Project 1, you should also use standard C debugging tools including `gdb` and `Valgrind`, which are also installed on the containers.

4.6 Automatic code formatting and analysis

We provided a `pre-commit` [14] configuration file that you can use to automatically format and statically check your code whenever you run `make format` inside one of the containers. You can check `.pre-commit-config.yaml` to see all the hooks that we run. You can optionally also install it in your development machine. This will ensure that it always runs all the hooks whenever you create a new commit. If you want to install `pre-commit` in your local machine, run `pip3 install pre-commit`. Then run `pre-commit install` in the root directory of the `cmu-tcp` repository to install the pre-commit hooks.

5 Logistics

Starter code. We reserve the right to change the starter code as the project progresses to fix bugs and to introduce new features that will help you debug your code (we commit to making the changes as transparent as possible so as to not break your working implementation). You are responsible for checking Ed to stay up-to-date on these changes. We will assume that all students in the class will read and be aware of any information posted to Ed.

Reusing code. You are permitted to use any code you have previously written (e.g., from 15-213/513, a hackathon, etc.), but you may not use code from anyone or anywhere else. You are welcome to clone the project repository to stay up-to-date with changes (e.g., bug-fixes), but please do not push your solution code to a public fork. We take academic integrity violations *very* seriously, and you will be held just as liable for sharing code with someone as you will be for using it.

Testing. The starter code includes a suite of public test-cases to help you get started, but the autograder may use private test-cases to grade your solution (you will see the final score at the time of submission, though). Please come up with your own tests; it'll save both you and your partner a lot of debugging pain in the long term!

References

[1] Textbook Section 5.2 Reliable Byte Stream (TCP):
https://book.systemsapproach.org/e2e/tcp.html 2

[2] Textbook Section 6.3 TCP Congestion Control:
https://book.systemsapproach.org/congestion/tcpcc.html 2, 3.2.1

[3] TCP connection establishment and termination:
https://book.systemsapproach.org/e2e/tcp.html#connection-establishment-and-termination 1, 5

[4] TCP sliding window:
https://book.systemsapproach.org/direct/reliable.html#sliding-window
https://book.systemsapproach.org/e2e/tcp.html#sliding-window-revisited 3

[5] `tcconfig`: https://github.com/thombashi/tcconfig 4.2

[6] `tcpdump`: https://linux.die.net/man/8/tcpdump 4.3

[7] `tshark`: https://www.wireshark.org/docs/man-pages/tshark.html 4.3

[8] Creating a Wireshark dissector in Lua: https://mika-s.github.io/wireshark/lua/dissector/2017/11/04/creating-a-wireshark-dissector-in-lua-1.html 4.3

[9] Wireshark plugin folder:
https://www.wireshark.org/docs/wsug_html_chunked/ChPluginFolders.html 4.3

[10] Vagrant: https://www.vagrantup.com/intro/getting-started/index.html 4.1

[11] Docker Desktop: https://docs.docker.com/desktop/ 4.1

[12] Git Basics – Tagging: https://git-scm.com/book/en/v2/Git-Basics-Tagging 1

[13] `pytest`: https://docs.pytest.org/en/stable/ 4.5

[14] `pre-commit`: https://pre-commit.com/ 4.6

[15] James Kurose and Keith Ross. 2016. *Computer networking: A top-down approach* (7th ed.). Pearson, Upper Saddle River, NJ. 2